

网络技术与应用实验报告

实验3：通过编程获取IP地址与MAC地址的对应关系

姓名：谢小珂 学号：2310422 专业：物联网工程

一、实验要求

通过编程获取IP地址与MAC地址的对应关系实验，要求如下：

- (1) 在IP数据报捕获与分析编程实验的基础上，学习WinPcap的数据包发送方法。
- (2) 通过WinPcap编程，获取IP地址与MAC地址的映射关系。
- (3) 程序要具有输入IP地址，显示输入IP地址与获取的MAC地址对应关系界面。界面可以是命令行界面，也可以是图形界面，但应以简单明了的方式在屏幕上显示。
- (4) 编写的程序应结构清晰，具有较好的可读性。

二、实验环境与配置

2.1 Npcap / Npcap SDK 安装

为实现数据包捕获，需要安装 **Npcap 驱动**与 **Npcap SDK**：

1. 访问 Npcap 官网并下载最新版本安装包

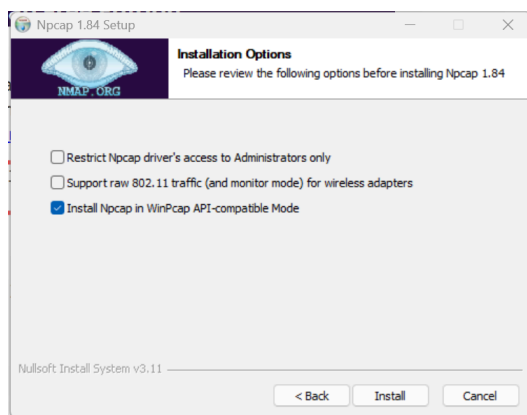
Downloading and Installing Npcap Free Edition

The free version of Npcap may be used (but not externally redistributed) on up to 5 systems ([free license details](#)). It may also be used on unlimited systems where it is only used with [Nmap](#), [Wireshark](#), and/or [Microsoft Defender for Identity](#). Simply run the executable installer. The full source code for each release is available, and developers can build their apps against the SDK. The improvements for each release are documented in the [Npcap Changelog](#).

- [Npcap 1.84 installer](#) for Windows 7/2008R2, 8/2012, 8.1/2012R2, 10/2016, 2019, 11 (x86, x64, and ARM64).
- [Npcap SDK 1.15](#) (ZIP).
- [Npcap 1.84 debug symbols](#) (ZIP).
- [Npcap 1.84 source code](#) (ZIP).

The latest development source is in our [Github source repository](#). Windows XP and earlier are not supported; you can use [WinPcap](#) for these versions.

2. 安装时勾选：**Install Npcap in WinPcap API-compatible Mode**

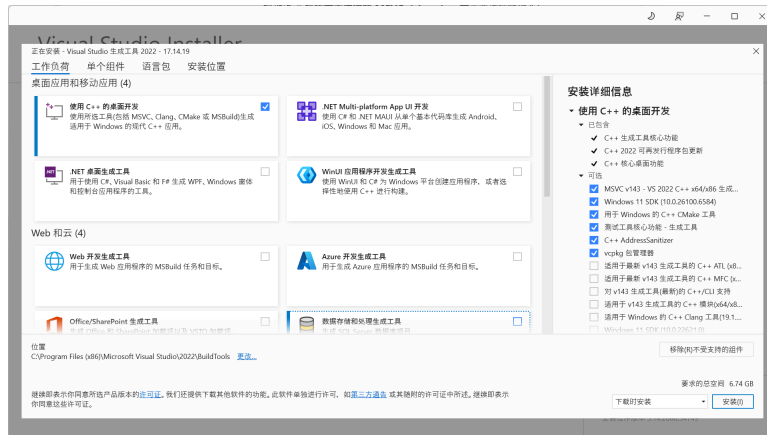


3. 下载 Npcap SDK，解压后可看到两个关键目录：

- | | | |
|---|---------|-----------------------|
| 1 | Include | → 头文件目录 |
| 2 | Lib | → 库文件目录 (x64 / win32) |

2.2 组件与安装

- **编辑器**: Visual Studio Code
- **编译器**: **Microsoft Visual C++ (MSVC)**, 通过 **Visual Studio Build Tools** 安装
 - 工作负载: 使用 C++ 的桌面开发 (含 MSVC v143、Windows SDK、CMake)



- **抓包驱动与 SDK**: **NPCap** (运行时) + **NPCap SDK** (开发头/库)
- **说明**: 本实验选用 **x86 (32 位)** 编译目标, 与当前 `wpcap.lib` / `Packet.lib` 的库架构一致, 避免 “x86/x64 不匹配” 的链接错误。

安装完成后, 将 NPCap SDK 中的 **Include** 与 **Lib** 目录拷贝到工程根目录, 形成本地依赖, 避免全局路径配置。

2.3 NPCap SDK 配置 (不配置环境变量, 直接放项目目录)

将 NPCap SDK 解压后, 把里面的 **Include** 和 **Lib** 文件夹直接放到实验工程目录:

```
1 | ARP_project/
2 | | arp_lookup.cpp    ← 实验代码
3 | |
4 | |─Include/          ← SDK 头文件 (pcap.h)
5 | |─Lib/              ← SDK 库文件 (wpcap.lib / Packet.lib)
```

优点: 路径不用写绝对路径, 也不用手动配置系统环境变量。

三、实验过程

3.1 WinPcap 数据包发送方法在本程序中的实现

本实验程序基于 WinPcap/NPCap 提供的 `pcap` 接口, 在用户态手工构造以太网帧并调用 `pcap_sendpacket()` 将其发送到链路层。整个发送过程可以概括为:

初始化环境 → 发现并选择网卡 → 打开网卡获得 `pcap_t` 句柄 → 获取本机 IP/MAC 与目标 IP → 构造 ARP 请求帧 → 用 `pcap_sendpacket()` 多次发送

1. 发送相关模块与依赖配置

程序源文件 `arp_lookup.cpp` 首先包含了 `Winsock2`、`pcap.h` 以及标准 C 库头文件，并通过

```
1  #pragma comment(lib, "wpcap.lib")
2  #pragma comment(lib, "Packet.lib")
3  #pragma comment(lib, "ws2_32.lib")
```

在链接阶段显式依赖 WinPcap 兼容库 `wpcap.lib` / `Packet.lib` 和 Windows 套接字库 `ws2_32.lib`，确保后续可以调用 `pcap_findalldevs()`、`pcap_open_live()`、`pcap_sendpacket()` 等接口以及 `inet_pton()` 等地址转换函数。

在正式使用 WinPcap 之前，程序调用

```
1  WSADATA wsd;
2  WSStartup(MAKEWORD(2,2), &wsd);
```

初始化 Winsock 运行环境，如果返回值不为 0 则直接终止。这样可以保证所有基于 Winsock 的 API（例如 `inet_pton`）处于可用状态，为后续构造报文时的 IP 地址转换提供底层支持。

2. 网卡枚举与发送句柄获取

WinPcap 的所有发送和接收操作都依赖于 `pcap_t*` 类型的会话句柄。本程序通过 `pcap_findalldevs()` 枚举本机所有可用网卡，再由用户选择具体用于发送的接口：

1. 调用

```
1  pcap_if_t *alldevs = NULL;
2  char errbuf[PCAP_ERRBUF_SIZE] = {0};
3  if (pcap_findalldevs(&alldevs, errbuf) == -1) { ... }
```

获取网卡链表 `alldevs`。若调用失败或返回列表为空，则输出错误信息并清理 Winsock 资源后退出。

2. 遍历 `alldevs`，为每块网卡打印编号、设备名和描述信息：

```
1  for (pcap_if_t *d = alldevs; d; d = d->next) {
2      printf("[%d] %s (%s)\n", ++idx, d->name,
3          d->description ? d->description : "No description");
4  }
```

3. 通过键盘输入编号 `choice`，利用循环将指针移动到选中的 `pcap_if_t *dev`，若编号不在合法范围内则认为输入无效并退出。

选定网卡后，程序调用 WinPcap 的实时打开接口获取发送句柄：

```

1  pcap_t *handle = pcap_open_live(
2      dev->name,      // 接口名
3      65536,          // snaplen, 足够容纳完整以太网帧
4      1,              // 启用混杂模式
5      1000,           // 超时 1000 ms
6      errbuf
7  );

```

如果 `handle` 为 `NULL`，说明网卡打开失败，程序打印错误信息并释放设备链表、清理 Winsock 环境后返回。成功时，`handle` 作为后续 `pcap_sendpacket()` 的第一个参数，代表“与目标网卡的发送会话”。

3. 以太网与 ARP 帧结构的定义与布局

为了向 WinPcap 发送完整的二进制帧，程序没有依赖操作系统协议栈自动封装，而是自己定义了以太网首部和 ARP 报文首部的数据结构：

```

1  #pragma pack(push,1)
2  typedef struct {
3      u_char   dst_mac[6];
4      u_char   src_mac[6];
5      uint16_t eth_type;    // 以太网类型，网络字节序
6  } eth_hdr_t;
7
8  typedef struct {
9      uint16_t htype;        // 硬件类型，1 表示以太网
10     uint16_t ptype;        // 上层协议类型，0x0800 表示 IPv4
11     u_char   hlen;         // 硬件地址长度（MAC 为 6）
12     u_char   plen;         // 协议地址长度（IPv4 为 4）
13     uint16_t oper;         // 操作码：1=请求，2=应答
14     u_char   sha[6];       // 发送端 MAC
15     uint32_t spa;          // 发送端 IP（网络字节序）
16     u_char   tha[6];       // 目标 MAC
17     uint32_t tpa;          // 目标 IP（网络字节序）
18 } arp_pkt_t;
19
20 typedef struct {
21     eth_hdr_t eth;
22     arp_pkt_t arp;
23 } arp_frame_t;
24 #pragma pack(pop)

```

通过 `#pragma pack(push,1)` / `#pragma pack(pop)` 将结构体对齐方式固定为 1 字节，避免编译器插入填充字节，保证内存中的字段排列与 ARP 以太网帧的实际格式完全一致；同时对所有 16 位字段（如 `eth_type`、`htype`、`ptype`、`oper`）统一使用 `htons()` 转换为网络字节序。

这一设计使得程序可以直接对 `arp_frame_t` 取 `sizeof` 并作为帧长度传入 `pcap_sendpacket()`，从而以“结构体即报文”的方式进行发送。

4. 发送端参数的采集与 ARP 请求帧构造

为构造与实际网络环境匹配的 ARP 请求，程序通过命令行交互获取本机和目标主机信息：

1. **本机 IP 地址**：提示用户输入如 192.168.129.1 的字符串，调用 `parse_ip()`（内部封装 `inet_pton`）转换为网络字节序的 `uint32_t`，存入 `local_ip_net`。
2. **本机 MAC 地址**：提示按 AA-BB-CC-DD-EE-FF 形式输入，通过 `sscanf()` 解析为 6 个字节写入 `local_mac` 数组；若格式错误则立即报错退出。程序预留了输入 00-00-00-00-00-00 时自动获取 MAC 的逻辑，但出于简化考虑当前未实现，仍要求用户手工填写。
3. **目标 IP 地址**：进一步提示输入需要解析的目标 IP 字符串，转换为网络字节序后保存在 `target_ip_net` 中，用于 ARP 报文字段 `tpa`。

在上述参数基础上，程序创建 `arp_frame_t frame` 并逐字段填充：

- **以太网首部**
 - `dst_mac`：全部置为 0xFF，表示目的 MAC 为广播地址；
 - `src_mac`：拷贝自用户输入的本机 MAC；
 - `eth_type`：设置为 `htons(0x0806)`，即上层协议为 ARP。
- **ARP 报文首部**
 - `htype = htons(1)` (Ethernet)，`ptype = htons(0x0800)` (IPv4)；
 - `hlen = 6`，`plen = 4`；
 - `oper = htons(1)`，表示当前为 ARP 请求报文；
 - `sha / spa`：分别填入本机 MAC、本机 IP；
 - `tha`：在请求中置为全 0，目标 MAC 未知；
 - `tpa`：写入待解析的目标 IP 地址。

完成以上填充后，`frame` 即为一帧完整的 ARP Request 以太网帧，可直接发送至链路层。

5. 调用 `pcap_sendpacket()` 下发原始数据帧

WinPcap 提供的核心发送函数为：

```
1 | int pcap_sendpacket(pcap_t *p, const u_char *buf, int size);
```

本程序采用简单的多次重发策略：

```
1 | int send_count = 3;
2 | for (int i = 0; i < send_count; ++i) {
3 |     if (pcap_sendpacket(handle,
4 |                           (const u_char*)&frame,
5 |                           sizeof(frame)) != 0) {
6 |         printf("pcap_sendpacket error: %s\n", pcap_geterr(handle));
7 |     }
8 | }
9 | printf("ARP requests sent, waiting for reply...\n");
```

其中，`handle` 为前文通过 `pcap_open_live()` 打开的网卡句柄，`&frame` 指向已按协议格式构造好的 ARP 以太网帧，`sizeof(frame)` 为实际帧长。发送失败时，程序通过 `pcap_geterr(handle)` 输出详细错误原因，便于定位问题。

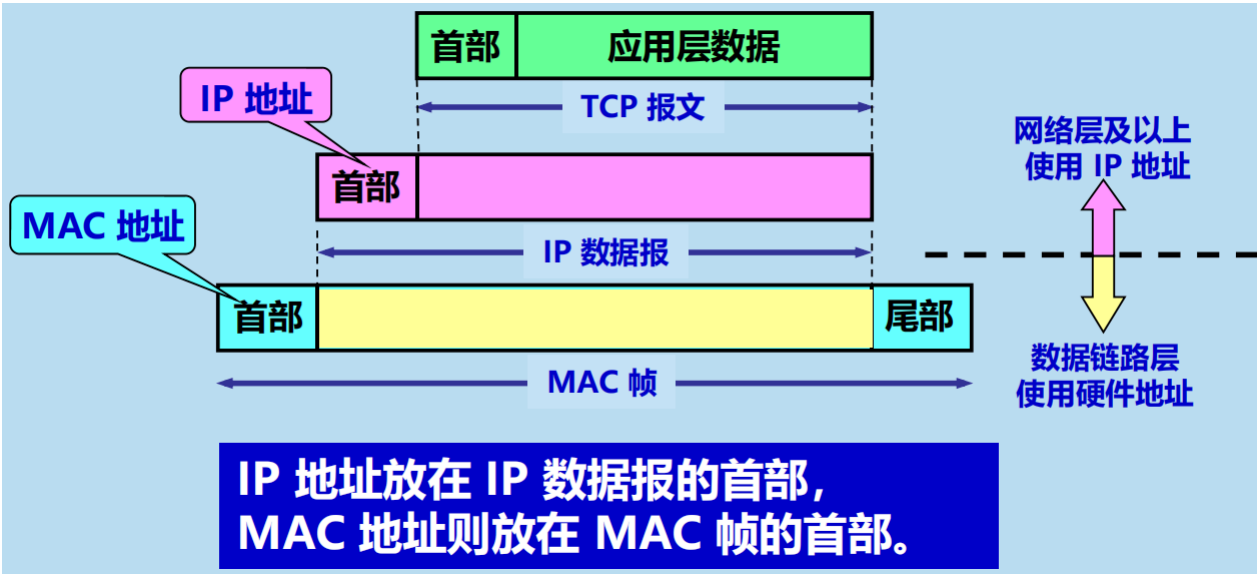
通过这种“手工构造帧头 + 原始帧发送”的方式，本程序完成了 WinPcap 数据包发送方法的实践：应用层直接对以太网帧进行字节级控制，并绕过操作系统协议栈，将 ARP 请求报文精确地下发到指定网卡，为后续 IP-MAC 映射关系的获取奠定了基础。

3.2通过 WinPcap 编程获取 IP 地址与 MAC 地址映射关系

在掌握了 WinPcap 的数据包发送方法之后，本实验利用 ARP 协议的工作机制，在用户态主动发起 ARP 请求并捕获 ARP 应答，从而获得“指定目标 IP 对应的 MAC 地址”，即完成 IP 地址与 MAC 地址映射关系的编程实现。

1. IP-MAC 映射的实现思路

在以太网中，实际转发使用的是 MAC 地址，而 IP 地址则由上层协议使用。ARP 协议负责在同一局域网内将 IP 地址解析为 MAC 地址：当本机需要向某个目标 IP 发送数据而 ARP 缓存中不存在对应映射时，会广播一条 ARP Request，持有该 IP 的主机收到后回复 ARP Reply，其中包含自己的 MAC 地址。



如图 所示，IP 地址封装在网络层 IP 数据报的首部，MAC 地址封装在数据链路层以太网帧的首部。网络层及以上通过 IP 地址标识主机，真正往物理介质上发送帧时则必须使用 MAC 地址，因此需要通过 ARP 在两者之间建立映射关系。

本程序正是借助这一过程：

1. 由用户指定“待解析的目标 IP”；
2. 在选定网卡上广播 ARP 请求；
3. 利用 WinPcap 持续捕获数据帧，并从中筛选出目标主机返回的 ARP 应答；
4. 从应答报文中读出发送端 MAC 地址，将其视为 `<目标 IP, 目标 MAC>` 的映射结果并输出。

2. 交互式输入与映射查询目标的确定

由于不同机器的网卡配置各不相同，程序不直接从系统接口自动推导 IP/MAC，而是通过命令行界面让用户明确给出查询参数：

- **选择网卡：**按照前文步骤，首先选定用于发送/接收 ARP 报文的 WinPcap 接口；
- **输入本机 IP：**例如 `192.168.129.1`，转换为 `local_ip_net`，用于报文中的发送端 IP；

- **输入本机 MAC**：例如 `00-50-56-C0-00-08`，解析为 6 字节数组 `local_mac`，用于以太网首部和 ARP 报文的发送端 MAC；
- **输入目标 IP**：例如虚拟机的 `192.168.129.128`，转换为 `target_ip_net`，作为 ARP 报文的目标 IP，也作为后续筛选应答的依据。

通过这种方式，程序在运行时建立起“本机 IP/MAC + 目标 IP”的查询上下文：发送的 ARP Request 指向该目标 IP，而接收端则只关注源 IP 恰好为该目标的 ARP Reply。

3. 基于 WinPcap 捕获 ARP 应答报文

在前一节发送完 3 帧 ARP Request 之后，程序进入一个带总超时控制的循环，使用 `pcap_next_ex()` 从 WinPcap 会话中逐帧读取数据：

```

1  int timeout_ms      = 5000;      // 总超时 5 秒
2  int elapsed         = 0;
3  const int poll_interval = 100;   // 轮询间隔 100 ms
4  bool found          = false;
5
6  while (elapsed < timeout_ms && !found) {
7      struct pcap_pkthdr *pkt_header;
8      const u_char *pkt_data;
9      int res = pcap_next_ex(handle, &pkt_header, &pkt_data);
10     ...
11 }

```

其中：

- 返回值 `res == 0` 表示在设定的超时时间内没有捕获到任何数据包，此时程序调用 `Sleep(poll_interval)` 短暂休眠，再累加已等待时间后继续轮询；
- `res == 1` 表示成功捕获到一帧数据，程序对其执行协议解析与条件判断；
- `res == -1` 或 `res == -2` 分别表示发生错误或到达 EOF/抓包循环结束，此时程序打印错误信息后退出循环。

针对每一帧数据，程序首先检查其长度是否至少包含一个以太网首部和一个 ARP 报文首部，然后进行分层解析和筛选：

1. 将缓冲区开头解释为 `eth_hdr_t`，读取 `eth_type` 并转换字节序，只有 `eth_type == 0x0806` 的帧才可能是 ARP 报文，其它类型直接忽略；
2. 对于 ARP 帧，将 `pkt_data + sizeof(eth_hdr_t)` 位置解释为 `arp_pkt_t`，获取其中的 `oper` 和 `spa` 字段；
3. 仅当 `oper == 2` (ARP Reply) 且 `spa == target_ip_net` 时，才认为这是来自“目标 IP”的 ARP 应答。

这种双条件筛选（以太网类型 + ARP 操作码 + 发送端 IP）可以有效避免将本机向其他主机的 ARP 流量误判为本次查询的结果，保证了 IP-MAC 映射的唯一性和准确性。

4. 从 ARP 应答中解析 IP-MAC 映射结果

一旦匹配到符合条件的 ARP Reply，程序立即将其视为目标主机的响应并提取其中的 IP 和 MAC 信息：

```

1  if (op == 2 && rap->spa == target_ip_net) {
2      printf("Got ARP reply:\n");
3      printf("Sender IP: %s\n", target_ip_str);
4      printf("Sender MAC: ");
5      print_mac(rap->sha);
6      printf("\n");
7      found = true;
8      break;
9  }

```

这里：

- `rap->spa` 是 ARP 报文中发送端 IP 字段，已经在前述条件判断中确认与 `target_ip_net` 相等；
- `rap->sha` 是发送端 MAC 字段，调用 `print_mac()` 后以 `AA-BB-CC-DD-EE-FF` 的格式输出；
- `target_ip_str` 为用户输入的目标 IP 字符串，作为输出中的“Sender IP”字段。

因此，“输出的 Sender IP 和 Sender MAC”就构成了一条明确的 IP-MAC 映射 <目标 IP, 目标 MAC>。在实验中，可通过命令行 `ipconfig /all` 或在虚拟机端 `ifconfig` 查看目标主机 NIC 的 MAC 地址，与程序输出结果进行对比验证其正确性。

如果在 5 秒总超时时间内始终未捕获到符合条件的 ARP Reply，程序会输出：

```

1  if (!found) {
2      printf("No ARP reply received within %d ms\n", timeout_ms);
3  }

```

提示可能是目标 IP 不在同一网段、目标主机关闭、网络不通或被防火墙过滤等原因导致解析失败。

小结：基于 ARP 的 WinPcap IP-MAC 解析流程

综上，本程序通过如下步骤实现了“通过编程获取 IP 地址与 MAC 地址映射关系”的实验目标：

1. 利用 WinPcap/NPcap 枚举网卡、打开指定接口并构造 ARP Request 以太网帧；
2. 通过 `pcap_sendpacket()` 在链路层广播目标 IP 的 ARP 请求；
3. 依靠 `pcap_next_ex()` 持续捕获数据包，并在用户态按字节解析以太网头和 ARP 报文头；
4. 通过“以太网类型 + ARP 操作码 + 发送端 IP”多重条件精确筛选出目标主机发回的 ARP Reply；
5. 从应答中抽取 <spa, sha> 作为 <目标 IP, 目标 MAC> 映射条目，在命令行界面直观显示。

与直接查看操作系统 ARP 缓存不同，这一实现是一次“主动发起、用户态解析”的过程，使得 IP-MAC 映射关系的获取过程完全透明可见，也进一步加深了对 ARP 协议和 WinPcap 编程模型的理解。

3.3完整代码与注释

```

1  // arp_lookup.cpp
2  // 功能：在指定网卡上构造并发送 ARP 请求，通过捕获 ARP 应答获取目标 IP 的 MAC 地址。
3  // 编译示例 (Developer Command Prompt for VS):
4  // c1 /EHsc arp_lookup.cpp /I <npcap-sdk>\Include /link <libpath>\wpcap.lib
   // <libpath>\Packet.lib ws2_32.lib
5

```



```

6  #include <winsock2.h>      // Winsock2 基本网络接口 (WSAStartup、inet_pton 等)
7  #include <ws2tcpip.h>     // 补充 TCP/IP 相关函数
8  #include <pcap.h>         // WinPcap / Npcap 抓包与发包接口
9  #include <stdio.h>
10 #include <stdint.h>
11 #include <string.h>
12
13 #pragma comment(lib, "wpcap.lib")    // 链接 WinPcap / Npcap 捕获库
14 #pragma comment(lib, "Packet.lib")   // 链接底层 Packet 驱动库
15 #pragma comment(lib, "ws2_32.lib")   // 链接 Winsock2 库
16
17 // ----- 帮助函数 -----
18
19 // 以“AA-BB-CC-DD-EE-FF”的形式打印 6 字节 MAC 地址
20 void print_mac(const u_char *mac) {
21     printf("%02X-%02X-%02X-%02X-%02X-%02X",
22           mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
23 }
24
25 // 将点分十进制字符串 IP 转换为 uint32_t (网络字节序)
26 // 成功返回 0, 失败返回 -1
27 int parse_ip(const char *s, uint32_t *out_netorder) {
28     struct in_addr addr;
29     int r = inet_pton(AF_INET, s, &addr);
30     if (r != 1) return -1;
31     *out_netorder = addr.s_addr; // inet_pton 已经返回网络字节序
32     return 0;
33 }
34
35 // ----- 帧结构定义 (按字节严格布局) -----
36 // 使用 1 字节对齐, 保证结构体在内存中的布局与真实报文格式完全一致
37 #pragma pack(push,1)
38
39 // 以太网帧首部
40 typedef struct {
41     u_char  dst_mac[6];    // 目的 MAC
42     u_char  src_mac[6];    // 源 MAC
43     uint16_t eth_type;     // 以太网类型 (ARP 为 0x0806), 网络字节序
44 } eth_hdr_t;
45
46 // ARP 报文首部 (RFC 826)
47 typedef struct {
48     uint16_t htype;        // 硬件类型: 1 = Ethernet, 网络字节序
49     uint16_t ptype;        // 协议类型: 0x0800 = IPv4, 网络字节序
50     u_char  hlen;          // 硬件地址长度 (MAC 为 6)
51     u_char  plen;          // 协议地址长度 (IPv4 为 4)
52     uint16_t oper;         // 操作码: 1 = 请求, 2 = 应答, 网络字节序
53     u_char  sha[6];        // 发送端 MAC 地址
54     uint32_t spa;          // 发送端 IP 地址 (网络字节序)
55     u_char  tha[6];        // 目标 MAC 地址
56     uint32_t tpa;          // 目标 IP 地址 (网络字节序)
57 } arp_pkt_t;
58

```

```

59 // 完整 ARP 以太网帧: 以太网首部 + ARP 首部
60 typedef struct {
61     eth_hdr_t eth;
62     arp_pkt_t arp;
63 } arp_frame_t;
64
65 #pragma pack(pop)
66
67 // ----- 主体程序 -----
68 int main() {
69     // 1) 初始化 winsock (供 inet_pton 等函数使用)
70     WSADATA wsd;
71     if (WSAStartup(MAKEWORD(2,2), &wsd) != 0) {
72         printf("WSAStartup failed\n");
73         return 1;
74     }
75
76     // 2) 列出当前系统中可用的 Npcap / winPcap 接口, 让用户选择要使用的网卡
77     char errbuf[PCAP_ERRBUF_SIZE] = {0};
78     pcap_if_t *alldevs = NULL;
79     if (pcap_findalldevs(&alldevs, errbuf) == -1) {
80         printf("pcap_findalldevs failed: %s\n", errbuf);
81         WSACleanup();
82         return 1;
83     }
84
85     printf("==== Available Network Interfaces ==== \n");
86     int idx = 0;
87     for (pcap_if_t *d = alldevs; d; d = d->next) {
88         printf("[%d] %s (%s)\n", ++idx,
89             d->name,
90             d->description ? d->description : "No description");
91     }
92     if (idx == 0) {
93         printf("No interfaces found.\n");
94         pcap_freealldevs(alldevs);
95         WSACleanup();
96         return 1;
97     }
98
99     // 选择要使用的网卡编号
100     int choice = 0;
101     printf("Select interface index: ");
102     scanf("%d", &choice);
103     if (choice < 1 || choice > idx) {
104         printf("Invalid choice\n");
105         pcap_freealldevs(alldevs);
106         WSACleanup();
107         return 1;
108     }
109
110     // 找到选中的设备节点
111     pcap_if_t *dev = alldevs;

```

```

112     for (int i = 1; i < choice; ++i) dev = dev->next;
113
114     printf("Opening device: %s\n", dev->name);
115     // 3) 打开该设备: 抓取长度 65536 字节, 混杂模式, 超时 1000ms
116     pcap_t *handle = pcap_open_live(dev->name, 65536, 1, 1000, errbuf);
117     if (!handle) {
118         printf("pcap_open_live failed: %s\n", errbuf);
119         pcap_freealldevs(alldevs);
120         WSACleanup();
121         return 1;
122     }
123
124     // alldevs 链表已经不再需要, 释放之
125     pcap_freealldevs(alldevs);
126
127     // 4) 让用户输入本机 IP (字符串形式), 避免在不同 windows 环境下自动获取的兼容性问题
128     char local_ip_str[64];
129     printf("Enter your local interface IP (e.g. 192.168.1.10): ");
130     scanf("%s", local_ip_str);
131
132     uint32_t local_ip_net;
133     if (parse_ip(local_ip_str, &local_ip_net) != 0) {
134         printf("Invalid IP\n");
135         pcap_close(handle);
136         WSACleanup();
137         return 1;
138     }
139
140     // 5) 获取本机 MAC: 本程序采用“用户手工输入”的方式, 最直观可靠
141     char local_mac_input[32];
142     printf("Enter your local MAC (format: AA-BB-CC-DD-EE-FF) or 00-00-00-00-00-00 to
auto (try to auto): ");
143     scanf("%s", local_mac_input);
144
145     u_char local_mac[6] = {0};
146     bool mac_provided = false;
147
148     if (strcmp(local_mac_input, "00-00-00-00-00-00") != 0) {
149         // 解析用户输入的 MAC 字符串
150         unsigned int b[6];
151         if (sscanf(local_mac_input, "%x-%x-%x-%x-%x-%x",
152             &b[0], &b[1], &b[2], &b[3], &b[4], &b[5]) == 6) {
153             for (int i = 0; i < 6; ++i) local_mac[i] = (u_char)b[i];
154             mac_provided = true;
155         } else {
156             printf("Bad MAC format\n");
157             pcap_close(handle);
158             WSACleanup();
159             return 1;
160         }
161     } else {
162         // 预留: 可用 Packet.dll 自动获取本机 MAC (本实验为简化未实现)

```

```

163         printf("Auto MAC lookup not implemented in this demo. Please provide MAC
next time.\n");
164         pcap_close(handle);
165         WSACleanup();
166         return 1;
167     }
168
169     if (!mac_provided) {
170         printf("No valid MAC provided.\n");
171         pcap_close(handle);
172         WSACleanup();
173         return 1;
174     }
175
176     // 6) 询问要解析的“目标 IP”——最终需要得到该 IP 对应的 MAC
177     char target_ip_str[64];
178     printf("Enter target IP to resolve (e.g. 192.168.1.20): ");
179     scanf("%s", target_ip_str);
180
181     uint32_t target_ip_net;
182     if (parse_ip(target_ip_str, &target_ip_net) != 0) {
183         printf("Invalid target IP\n");
184         pcap_close(handle);
185         WSACleanup();
186         return 1;
187     }
188
189     // 7) 构造一帧完整的 ARP 请求报文（以太网帧 + ARP 首部）
190     arp_frame_t frame;
191     memset(&frame, 0, sizeof(frame));
192
193     // 以太网首部：目的 MAC 为广播 FF-FF-FF-FF-FF-FF，源 MAC 为本机，类型字段为 ARP (0x0806)
194     for (int i = 0; i < 6; ++i) frame.eth.dst_mac[i] = 0xFF;
195     for (int i = 0; i < 6; ++i) frame.eth.src_mac[i] = local_mac[i];
196     frame.eth.eth_type = htons(0x0806); // ARP
197
198     // ARP 首部：Ethernet + IPv4 + 请求报文
199     frame.arp.htype = htons(1);           // Ethernet
200     frame.arp.ptype = htons(0x0800);     // IPv4
201     frame.arp.hlen = 6;
202     frame.arp.plen = 4;
203     frame.arp.oper = htons(1);           // ARP request
204
205     // ARP 地址字段：本机作为发送端，目标 IP 为待解析主机
206     memcpy(frame.arp.sha, local_mac, 6); // 发送端 MAC
207     frame.arp.spa = target_ip_net;        // 发送端 IP
208     memset(frame.arp.tha, 0x00, 6);       // 请求中目标 MAC 填 0
209     frame.arp.tpa = target_ip_net;        // 目标 IP
210
211     // 8) 通过 WinPcap 多次发送 ARP 请求，以提高在不可靠链路上的成功率
212     int send_count = 3;
213     for (int i = 0; i < send_count; ++i) {
214         if (pcap_sendpacket(handle, (const u_char*)&frame, sizeof(frame)) != 0) {

```

```

215         printf("pcap_sendpacket error: %s\n", pcap_geterr(handle));
216     }
217 }
218 printf("ARP requests sent, waiting for reply...\n");
219
220 // 9) 使用 pcap_next_ex() 捕获 ARP 应答, 设置总超时 5 秒
221 int timeout_ms = 5000;          // 总超时时间
222 int elapsed    = 0;
223 const int poll_interval = 100;  // 每次等待 100ms
224 bool found = false;
225
226 while (elapsed < timeout_ms && !found) {
227     struct pcap_pkthdr *pkt_header;
228     const u_char *pkt_data;
229     int res = pcap_next_ex(handle, &pkt_header, &pkt_data);
230
231     if (res == 0) {
232         // 当前这次调用超时, 没有捕获到报文, 休眠一会儿后继续轮询
233         sleep(poll_interval);
234         elapsed += poll_interval;
235         continue;
236     } else if (res == 1) {
237         // 捕获到一帧报文, 先检查长度是否足够包含以太网首部 + ARP 首部
238         if (pkt_header->len < sizeof(eth_hdr_t) + sizeof(arp_pkt_t)) continue;
239
240         const eth_hdr_t *reh = (const eth_hdr_t*)pkt_data;
241         uint16_t ether_type = ntohs(reh->eth_type);
242         if (ether_type != 0x0806) continue; // 非 ARP 帧, 忽略
243
244         // 取出 ARP 部分进行进一步判断
245         const arp_pkt_t *rap = (const arp_pkt_t*)(pkt_data + sizeof(eth_hdr_t));
246         uint16_t op = ntohs(rap->oper);
247
248         // 只关心: ARP 应答 (op == 2) 且发送端 IP 等于目标 IP 的报文
249         if (op == 2 && rap->spa == target_ip_net) {
250             printf("Got ARP reply:\n");
251             printf("Sender IP: %s\n", target_ip_str);
252             printf("Sender MAC: ");
253             print_mac(rap->sha);    // 输出目标主机的 MAC
254             printf("\n");
255             found = true;
256             break;
257         }
258     } else if (res == -1) {
259         printf("pcap_next_ex error: %s\n", pcap_geterr(handle));
260         break;
261     } else if (res == -2) {
262         printf("pcap_next_ex: EOF or loop terminated\n");
263         break;
264     }
265 }
266
267 if (!found) {

```

```

268         printf("No ARP reply received within %d ms\n", timeout_ms);
269     }
270
271     // 10) 清理资源
272     pcap_close(handle);
273     WSACleanup();
274     return 0;
275 }

```

3.4 结果分析

步骤 1：确定本机网卡信息（IP 地址 + MAC 地址）

1. 在 Windows 打开 **命令提示符**，输入：

```
1 ipconfig /all
```

2. 找到用于发送 ARP 的网卡（例如无线网卡或 VMnet8），记录：

- **IPv4 地址：** 192.168.129.1
- **MAC 地址：** 00-50-56-C0-00-08

```

DHCPv6 IAID . . . . . : 134238294
DHCPv6 客户端 DUID . . . . . : 00-01-00-01-2C-51-02-08-08-BF-B8-C5-65-C2
TCP/IP 上的 NetBIOS . . . . . : 已启用

以太网适配器 VMware Network Adapter VMnet8:

   连接特定的 DNS 后缀 . . . . . :
   描述 . . . . . : VMware Virtual Ethernet Adapter for VMnet8
   物理地址. . . . . : 00-50-56-C0-00-08
   DHCP 已启用 . . . . . : 否
   自动配置已启用. . . . . : 是
   本地链接 IPv6 地址. . . . . : fe80::fe27:40de:2d4:3050%17(首选)
   IPv4 地址 . . . . . : 192.168.129.1(首选)
   子网掩码 . . . . . : 255.255.255.0
   默认网关. . . . . :
   DHCPv6 IAID . . . . . : 369119318
   DHCPv6 客户端 DUID . . . . . : 00-01-00-01-2C-51-02-08-08-BF-B8-C5-65-C2
   TCP/IP 上的 NetBIOS . . . . . : 已启用

无线局域网适配器 WLAN:

```

这些信息将在程序运行时输入。

步骤 2：编译 ARP 解析实验程序

在 **Developer Command Prompt for VS2022** 中切换到项目目录：

```

1 K:
2 cd K:\internet_grade3\ARP_project

```

编译：

```

1 cl /EHsc main.cpp arp_lookup.cpp /I include /link lib\wpcap.lib lib\Packet.lib
   ws2_32.lib

```

生成 **ARP.exe**。

步骤 3: 运行 ARP 请求程序, 发送 ARP 报文

在命令行执行:

```
1 | .\ARP.exe
```

程序自动:

1. 调用 `pcap_findalldevs()` 列出所有本地网卡 (Npcap接口)
2. 输入序号选择要使用的网卡

```
PS K:\internet_grade3\ARP_project> .\ARP.exe
==== Available Network Interfaces ====
[1] \Device\NPF_{18BC3F02-EF24-4F2F-A87E-A073D378153E} (WAN Miniport (Network Monitor))
[2] \Device\NPF_{90900E2D-CCF5-4BE6-8264-F981FDE24E77} (WAN Miniport (IPv6))
[3] \Device\NPF_{EE8086DD-B42D-43B0-81BB-95A61623712D} (WAN Miniport (IP))
[4] \Device\NPF_{34210448-D66C-49B1-B8D0-3CCA04A08337} (Hyper-V Virtual Ethernet Adapter)
[5] \Device\NPF_{492E4F9A-C958-4DEA-978F-1E7B36DBDF2E} (Bluetooth Device (Personal Area Network))
[6] \Device\NPF_{22664115-B708-43FE-A277-FBCAB31D0C18} (Intel(R) Wi-Fi 6 AX201 160MHz)
[7] \Device\NPF_{A7A6A6F5-0217-4064-8B7F-695554F1A7E0} (VMware Virtual Ethernet Adapter for VMnet8)
[8] \Device\NPF_{C566F866-7953-4C9E-8712-CD186A84FFEF} (VMware Virtual Ethernet Adapter for VMnet1)
[9] \Device\NPF_{8399018D-74F5-418D-B239-D8B410B0D86D} (Microsoft Wi-Fi Direct Virtual Adapter #2)
[10] \Device\NPF_{B8F704D8-1167-4157-9A81-F1F52DE67FC2} (Microsoft Wi-Fi Direct Virtual Adapter)
[11] \Device\NPF_{5D6AEB40-3A46-4FCA-B939-E3F319EFA4F1} (Famatech Radmin VPN Ethernet Adapter)
[12] \Device\NPF_{Loopback} (Adapter for loopback traffic capture)
[13] \Device\NPF_{EEF2BB7A-73AB-40BD-9233-8E9B29F3E2EB} (Sangfor SSL VPN CS Support System VNIC)
[14] \Device\NPF_{3128A90D-141E-4803-AAB5-85947099FA25} (Realtek PCIe GbE Family Controller)
Select interface index: 7
Opening device: \Device\NPF_{A7A6A6F5-0217-4064-8B7F-695554F1A7E0}
```

3. 输入:

- 本机 IP (`192.168.129.1`)
- 本机 MAC (`00-50-56-C0-00-08`) ——支持输入 `00-00-00-00-00-00` 自动获取

```
Opening device: \Device\NPF_{A7A6A6F5-0217-4064-8B7F-695554F1A7E0}
Enter your local interface IP (e.g. 192.168.1.10): 192.168.129.1
Enter your local MAC (format: AA-BB-CC-DD-EE-FF) or 00-00-00-00-00-00 to auto (try to auto): 00-50-56-C0-00-08
Enter target IP to resolve (e.g. 192.168.1.20):
```

4. 输入目标 IP, 实验虚拟机:

```
1 | 192.168.129.128
```

程序执行:

- ✓ 构造 ARP Request 报文
- ✓ 使用 `pcap_sendpacket()` 广播发送
- ✓ 使用 `pcap_next_ex()` 捕获 ARP Reply

输出:

```
[13] \Device\NPF_{EEF2BB7A-78AB-40BD-9233-8E9B29F3E2EB} (Sangfor SSL VPN CS Support System VNIC)
[14] \Device\NPF_{3128A90D-141E-4803-AAB5-85947099FA25} (Realtek PCIe GbE Family Controller)
Select interface index: 7
Opening device: \Device\NPF_{A7A6A6F5-0217-4064-8B7F-695554F1A7E0}
Enter your local interface IP (e.g. 192.168.1.10): 192.168.129.1
Enter your local MAC (format: AA-BB-CC-DD-EE-FF) or 00-00-00-00-00-00 to auto (try to auto): 00-50-56-C0-00-08
Enter target IP to resolve (e.g. 192.168.1.20): 192.168.129.128
ARP requests sent, waiting for reply...
Got ARP reply:
Sender IP: 192.168.129.128
Sender MAC: 00-0C-29-24-F6-42
PS K:\internet_grade3\ARP_project>
```

步骤 4：虚拟机端提供 ARP 回复（用于测试）

在 VMware Ubuntu 虚拟机终端执行：

```
1 | ifconfig
```

```
xxk@xxk-VMware-Virtual-Platform: ~/Desktop$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.129.128  netmask 255.255.255.0  broadcast 192.168.129.255
    inet6 fe80::20c:29ff:fe24:f642  prefixlen 64  scopeid 0x20<link>
    ether 00:0c:29:24:f6:42  txqueuelen 1000  (Ethernet)
    RX packets 40  bytes 15936 (15.9 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 112  bytes 16763 (16.7 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 128  bytes 11486 (11.4 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 128  bytes 11486 (11.4 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
```

找到虚拟机的：

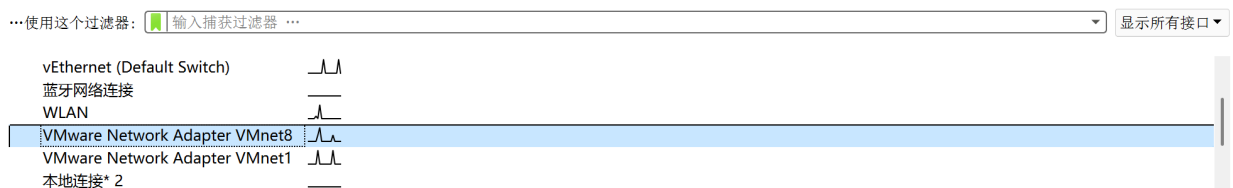
- **IPv4 地址：** 192.168.129.128
- **MAC 地址：** 00:0c:29:24:f6:42

确保虚拟机能够与宿主机 VMnet8 网卡互通。

步骤 5：使用 Wireshark 验证 ARP 报文交互

1. 打开 Wireshark → 选择对应的网卡 (如 VMnet8)

捕获



2. 在过滤器输入：

1. 可以看到：

- 主机发出的 **ARP Request**
- 虚拟机发出的 **ARP Reply**

14	43.039324	VMware_c0:00:08	Broadcast	ARP	42 Who has 192.168.129.128? Tell 192.168.129.1
15	43.039369	VMware_c0:00:08	Broadcast	ARP	42 Who has 192.168.129.128? Tell 192.168.129.1
16	43.039378	VMware_c0:00:08	Broadcast	ARP	42 Who has 192.168.129.128? Tell 192.168.129.1
17	43.039650	VMware_24:f6:42	VMware_c0:00:08	ARP	60 192.168.129.128 is at 00:0c:29:24:f6:42
18	43.039698	VMware_24:f6:42	VMware_c0:00:08	ARP	60 192.168.129.128 is at 00:0c:29:24:f6:42
19	43.039747	VMware_24:f6:42	VMware_c0:00:08	ARP	60 192.168.129.128 is at 00:0c:29:24:f6:42

证明 ARP 解析成功，且 MAC 地址与程序输出一致。

四、实验总结

1. 可以改进的地方

- **本机参数获取仍依赖手工输入。** 当前程序要求用户在命令行中输入本机 IP 和 MAC 地址，如果格式写错就需要重新运行。后续可以调用 Windows API 或 Packet.dll 自动枚举本机网卡的 IP 与 MAC 信息，并在界面中列出候选项供选择，减少人为出错。
- **功能范围仍局限于“单 IP 查询”。** 目前一次只能解析一个目标 IP 的 MAC 地址，适合做原理验证，但不便于对局域网进行整体扫描。后续可以改为对一个 IP 段循环构造 ARP 请求，并维护一个 `<IP, MAC>` 映射表，输出类似“ARP 扫描结果”的列表，实现一个简易的局域网主机发现工具。
- **异常处理和提示信息可进一步细化。** 例如对 `pcap_open_live`、`pcap_sendpacket`、`pcap_next_ex` 等不同错误分别给出“网卡打开失败”“驱动未安装”“未选择正确虚拟网卡”等更明确的提示，便于定位问题。

2. 遇到的困难及解决过程

- **抓不到 ARP 应答包。** 实验初期多次出现程序一直等待但没有收到 ARP Reply 的情况。排查过程中发现主要原因包括：选择了错误的网卡（没有选中 VMnet8）、虚拟机与宿主机不在同一网段，以及防火墙丢弃了 ARP 报文。通过在 Wireshark 中对目标接口设置 ARP 过滤、对照 IP 配置并暂时关闭防火墙，最后确认报文已正确发出并收到应答，程序输出的 MAC 地址也与虚拟机实际网卡信息相一致。该过程加深了我对“环境配置 + 抓包分析”在网络实验中重要性的认识。